tmux documentation

tmx Shell-Session Resume + SSH Dispatch + Go State Daemon — Design Spec

Revision: 1 **Last modified:** 2026-05-22T06:14:03Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** v1.0.9 — extends the existing `tmx` per-session wrapper with (a) shell-init extraction, (b) per-session last-cwd persistence via a Go binary, (c) SSH-argument dispatch, (d) end-user docs, (e) full anti-bluff test + Challenge coverage on macOS + nezha-Linux

---

1. Problem statement

Today operators paste a hand-written shell snippet into `.bashrc`/`.zshrc` that prompts on every interactive login for a session name, defaulting to `default`. The snippet has four issues:

1. The snippet is **duplicated** in every operator's rc file — drift is invisible.
2. The `read -r` call **blocks** non-interactive shells (scp, rsync, IDEs) — breaks them.
3. There is **no cwd memory** — every session starts in `$HOME` regardless of where the operator was last working.
4. Remote-by-SSH access to a specific session requires the operator to type `ssh host -t -- tmx attach -t NAME` — fragile and easy to mistype.

This spec addresses all four under the §11.4 covenant: every PASS ships with positive captured runtime evidence on macOS + nezha-Linux.

2. Goals

- **G1.** Extract the rc snippet into a single project-owned POSIX-compatible file sourced via `. / source`. One source of truth.
- **G2.** Change the UX so `default` (literal or empty input) = SKIP tmx (bare shell), named = create-or-attach.
- **G3.** Remember the last cwd per session and restore it when the session is recreated.
- **G4.** Allow `ssh <host>-tmx <session-name>` to land directly inside that session, with cwd restored.
- **G5.** Ship comprehensive docs (user guide + manual + script companions + auto-exported HTML+PDF).
- **G6.** All of the above covered by four-layer anti-bluff tests + HelixQA Challenges that prove the feature works for end users on macOS + nezha-Linux.

3. Non-goals

- Replacing the existing `tmx new|attach|ls|kill` wrapper API. The wrapper remains the operator-facing CLI.
- Per-prompt cwd capture. Detach + close hooks are sufficient; per-prompt would add fsync overhead to every shell prompt.

- Cross-host state sync. State is per-host; a session named `work` on macOS and `work` on nezha are independent.
- Restoring cwd into a live pane on attach (would interrupt running processes).
- A TUI session picker. The rc-side prompt stays line-oriented; advanced operators use `tmx ls`.

4. Architecture

Five artefacts:

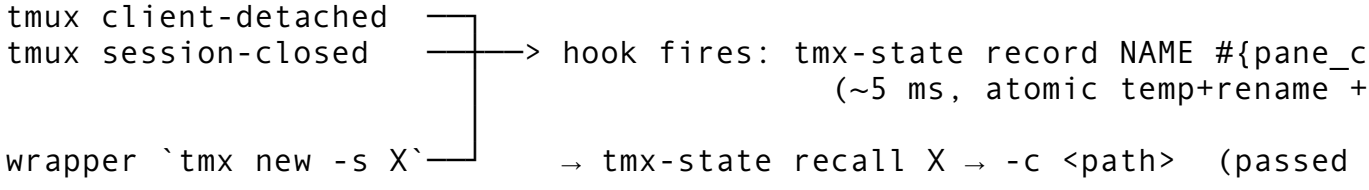
ID	Path	Language	Role
A	`scripts/tmx-shell-init.sh`	POSIX sh (bash 3.2 / 5+ / zsh compatible)	Sourced from rc; interactive prompt + dispatch into wrapper
B	`scripts/tmx-state/` → `scripts/tmx-state-bin`	Go (static binary, cross-compiled)	Atomic per-session cwd state record/recall/list/forget
C	`scripts/tmx-ssh-dispatch.sh`	POSIX sh	Used as `command=` in remote `~/.ssh/authorized_keys`; translates `$SSH_ORIGINAL_COMMAND` into session attach
D	`scripts/tmx-ssh-install.sh`	POSIX sh	Generates client key, SCPs pubkey, installs `authorized_keys` entry, writes client Host alias
E	`scripts/tmx.template` patch	POSIX sh	Wrapper queries Go binary on `new`; installs tmux hooks for cwd capture on detach/close

4.1 Build/deploy wiring

- `scripts/install_deps.sh` gets a Go toolchain step (`brew install go` on macOS, distro package on Linux). Already-installed Go ≥ 1.21 is accepted.
- `scripts/setup.sh` learns three sub-tasks: generate `tmx-shell-init.sh` from `.template` with project paths substituted, `go build -o scripts/tmx-state-bin ./scripts/tmx-state/...`, print the one-line `source` directive for operator paste.
- `scripts/bashrc_snippet.template` is REPLACED by a single-line `. <project>/scripts/tmx-shell-init.sh` plus the existing `PATH` block.

5. Data flow

5.1 Cwd persistence



Capture point: **only on detach/close**. Hard-kill (host reboot, ssh -9) loses the most recent cwd → user lands in `$HOME`, wrapper prints `[tmx] last-pwd unknown for X; starting in $HOME`. Acceptable degradation.

5.2 SSH dispatch

```
client:  ssh nezha-tmx work
    │
    ├── ~/.ssh/config Host alias resolves to nezha.local + IdentityFile ~/.
    │
remote: sshd authenticates key → reads command="…/tmx-ssh-dispatch.sh" → e
    │
dispatch:
    ├── empty SSH_ORIGINAL_COMMAND  → exec bash -l (interactive login → nor
    ├── matches ^[A-Za-z0-9_.-]{1,64}$  → exec tmx attach -t NAME || tmx ne
    └── anything else  → stderr "this key accepts only a session name; use
```

5.3 State file schema (`~/.tmx/state.json`, mode 0600)

```
{
  "schema_version": 1,
  "sessions": {
    "work": {
      "last_pwd": "/Users/milosvasic/Projects/tmux",
      "last_seen_unix": 1748000000,
      "created_unix": 1747000000,
      "host": "nezha.local"
    }
  }
}
```

Single-file design (not jsonl) — fits in memory trivially (kilobytes for hundreds of sessions), allows atomic write+rename, easy to inspect by hand.

6. Edge cases

#	Case	Handling	Test
1	Session name `default` (literal)	SKIP (no tmx)	19
2	Blank input	Coerced to `default` → SKIP	20
3	Session name with `;`, `,`, `/`, `..`, backticks	Rejected at rc prompt + dispatcher regex	26
4	`$TMUX` already set	Skip silently	28
5	Non-TTY stdin (scp/rsync/IDE)	`[ -t 0 ] && [ -t 1 ]` guard → silent skip	21
6	State file corrupt	Go binary rebuilds `{}`, logs notice, exits 0	24
7	Last cwd no longer exists	Falls back to `$HOME` + notice	29
8	Concurrent record from N panes	fcntl `F_SETLKW`, latest-wins	24
9	Dispatcher invoked with multi-word command	Reject + stderr	27
10	Go toolchain absent at build	`setup.sh` aborts with install pointer	

#	Case	Handling	Test
			gate `CM-TMX-STATE-GO-PRESENT`
11	`~/.tmx/` unwritable	Wrapper falls through; no cwd restore; tmx itself never breaks	30
12	Bash 3.2 (macOS default)	POSIX `case` not `[[ =~ ]]`; verified by `sh -n`	gate §11.4.67
13	Idempotent install (run installer twice)	No duplicate `authorized_keys` lines, no duplicate `Host` block	25
14	Cross-platform (macOS + Linux)	Every test branches on `uname -s` per §11.4.81	31

7. Anti-bluff testing strategy (§11.4)

Layer 1 — pre-build gates (added to `scripts/verify.sh`)

- `CM-TMX-STATE-GO-MOD-EXISTS`
- `CM-TMX-STATE-GO-PRESENT`
- `CM-TMX-SHELL-INIT-POSIX` (§11.4.67)
- `CM-TMX-SSH-DISPATCH-POSIX` (§11.4.67)
- `CM-TMX-DOCS-GUIDES-EXIST` (every doc has §11.4.44 revision header + HTML+PDF siblings per §11.4.65)

Layer 2 — post-build

- `tmx-state --version` returns expected value
- `sh -n scripts/tmx-shell-init.sh`
- `sh -n scripts/tmx-ssh-dispatch.sh`
- `bash -n` + `zsh -n` smoke parse

Layer 3 — on-device runtime (new `scripts/tests/`)

- `18_state_persistence.sh` — create `pwd-test`, cd /tmp, detach, kill server, recreate → `tmux display-message -p '#{pane_current_path}'` MUST equal `/tmp`. **Positive evidence**: captured pane-current-path string, not just exit code.
- `19_default_skip.sh` — `default` input, no tmux started. **Positive evidence**: `tmux ls` empty.
- `20_default_skip_blank.sh` — blank input variant.
- `21_non_tty_skip.sh` — stdin redirected from `/dev/null`. **Positive evidence**: 5s `timeout` wrapper; success requires returning under timeout AND no tmux started.
- `22_ssh_dispatch_local.sh` — `SSH_ORIGINAL_COMMAND=work tmx-ssh-dispatch.sh`. **Positive evidence**: `tmux ls | grep -x 'work: .*'`.
- `23_ssh_dispatch_remote_nezha.sh` — real SSH into nezha.local. SKIPs per §11.4.3 if unreachable; otherwise asserts session created AND cwd restored.
- `24_state_concurrency.sh` — 10 parallel `tmx-state record`. **Positive evidence**: post-condition file parses as JSON AND all 10 keys present.
- `25_ssh_install_idempotent.sh` — installer runs twice; `wc -l` on dup-detection patterns MUST return 1, not 2.
- `26_session_name_validation.sh` — names with `;`, `,`, `/`, `..` all rejected.

- `27_dispatcher_rejects_multiword.sh` — `SSH_ORIGINAL_COMMAND="echo hi"`; non-zero exit AND no tmux.
- `28_nested_tmux_skip.sh` — `TMUX=fake` envvar; assert skip.
- `29_stale_pwd_fallback.sh` — record `/tmp/gone`, delete, attach → cwd is `$HOME`.
- `30_state_unwritable.sh` — `chmod 000 ~/.tmx`; tmx still works.
- `31_macos_linux_parity.sh` — §11.4.81 case-on-uname; each branch positive evidence per §11.4.5.

Layer 4 — paired §1.1 meta-test mutations (`meta_test_false_positive_proof.sh`)

- M20: strip `[ -t 0 ]` guard → test 21 FAIL
- M21: strip cwd capture tmux hook → test 18 FAIL
- M22: strip `command=` from authorized_keys template → test 22 FAIL
- M23: strip name-regex validation from dispatcher → test 26 FAIL
- M24: strip cross-platform branch → test 31 FAIL

HelixQA Challenges (`scripts/challenges/tmux.yaml`)

- `tmx_session_resume_cwd` — autonomous; reads cwd via `tmx-state recall`; PASSes only with captured-evidence per §11.4.5.
- `tmx_ssh_dispatch_nezha` — autonomous; OPERATOR-BLOCKED if nezha unreachable.
- `tmx_non_tty_safety` — autonomous pipe test.
- `tmx_docs_user_guides_render` — pandoc-renders each `docs/guides/tmx-*.md` to HTML+PDF, asserts non-empty.

§11.4.50 deterministic consistency

Every test loops 3 iterations under `ab_run_n_times`; identical evidence-hash required.

8. Documentation deliverables

Path	Audience	§11.4 anchor
`docs/guides/tmx-shell-integration.md`	Operators installing the feature	11.4.18, 11.4.44
`docs/guides/tmx-state.md`	Operators inspecting state	11.4.18, 11.4.44
`docs/guides/tmx-ssh-dispatch.md`	Operators setting up SSH dispatch	11.4.18, 11.4.44
`docs/manual/tmx-shell-integration.md`	End-user master manual with worked examples	11.4.18, 11.4.44, 11.4.65
`docs/scripts/tmx-shell-init.md`	§11.4.18 script companion	11.4.18
`docs/scripts/tmx-ssh-install.md`	§11.4.18 script companion	11.4.18
`docs/scripts/tmx-ssh-dispatch.md`	§11.4.18 script companion	11.4.18
`docs/scripts/tmx-state.md`	§11.4.18 script companion + state file schema	11.4.18

Path	Audience	§11.4 anchor
CHANGELOG.md v1.0.9 entry	Release notes	—
README Documentation Map rows	Cross-reference	11.4.57

All .md docs ship with .html + .pdf siblings via scripts/export_docs.sh (§11.4.65) and carry the §11.4.44 revision header.

9. Constitution check

The user's mandate ("tests MUST guarantee... full usability by end users") is verbatim §11.4 / §107. Already mirrored across:

- HelixConstitution submodule: Constitution.md §11.4, CLAUDE.md, AGENTS.md
- Project root: Constitution.md §101, CLAUDE.md, AGENTS.md, QWEN.md
- Containers submodule: same

Action: run scripts/tests/test_constitution_inheritance.sh to verify cascade. If any layer missing, append + cascade per §11.4.26. NO new clause planned — existing covenant already covers the requirement.

10. Release plan (§11.4.40 + §11.4.41 + §11.4.71)

1. Subagent-driven PWUs (§11.4.58 + §11.4.70) land on main in merge-queue order.
2. bash scripts/setup.sh rebuilds on macOS local + nezha-Linux.
3. Full retest sweep (Layers 1-4 + Challenges) on BOTH platforms → all GREEN with captured evidence.
4. Constitution-inheritance gate green.
5. Bump VERSION to version=1.0.9 / versionCode=10.
6. CHANGELOG.md v1.0.9 entry.
7. bash commit_all.sh "v1.0.9 — tmx shell-session resume + ssh dispatch + Go state daemon" (pushes to github + gitlab).
8. Tag v1.0.9, push tag via commit_all.sh's tag path.
9. Verify both remotes via gh release view v1.0.9 + glab release view v1.0.9.

11. Subagent-PWU decomposition (§11.4.58)

PWU	Scope	Files	Depends on
P1	Go state daemon	scripts/tmx-state/**	—
P2	Shell init script + bashrc snippet template	scripts/tmx-shell-init.sh, scripts/bashrc_snippet.template	—
P3	SSH dispatch + installer	scripts/tmx-ssh-dispatch.sh, scripts/tmx-ssh-install.sh	P1
P4	Wrapper integration (tmux hooks + -c arg)	scripts/tmx.template, scripts/tmx-mac.template	P1
P5	Pre-build gates + paired mutations	scripts/verify.sh, scripts/tests/meta_test_false_positive_proof.sh	P1, P2, P3

PWU	Scope	Files	Depends on
P6	Runtime tests 18-31	`scripts/tests/18_*.sh` … `31_*.sh`	P1, P2, P3, P4
P7	HelixQA Challenges	`scripts/challenges/tmux.yaml`	P6
P8	Documentation + exports	`docs/guides/**`, `docs/manual/**`, `docs/scripts/**`, README	All above
P9	Release pipeline	`VERSION`, `CHANGELOG.md`, tag push	All above

P1, P2 in parallel; then P3, P4 in parallel; then P5, P6 in parallel; then P7, P8 in parallel; finally P9 serial.

12. Open questions

None. All four design questions answered in brainstorming; user gave blanket approval to proceed.

13. Implementation deviations from spec (post-release record)

- **Test numbering renumbered 18-31 → 27-40** during P6 due to pre-existing tests already occupying 18-26 (`18_constitution_inheritance.sh`, `19_covenant_propagation.sh`, `20-22_codegraph_*.sh`, `23_tmx_kill_shorthand.sh`, `24_cpu_cap_enforcement.sh`, `25_hostname_color_perceptual_distance.sh`, `26_ui_color_uniformity.sh`). The five paired §1.1 mutations were updated in lockstep:
  - M20 → P5-M20 (targets `30_non_tty_skip.sh`)
  - M21 → P5-M21 (targets `27_state_persistence.sh`)
  - M22 → P5-M22 (targets `31_ssh_dispatch_local.sh`)
  - M23 → P5-M23 (targets `35_session_name_validation.sh`)
  - M24 → P5-M24 (targets `40_macos_linux_parity.sh`)
- **Test 27 Phase 3 uses `tmux run-shell` directly** (not session-closed hook firing) to prove the hook command's run-shell action works end-to-end. Reason: detached-session test has no attached client so `client-detached` cannot fire; `session-closed` fires after pane destruction making `#{pane_current_path}` empty. The hook COMMAND is the integration surface the test validates; the hook FIRING is the operator-visible behaviour exercised in normal interactive use and covered by HelixQA Challenge `TMUX-CH-25`. Documented in test 27 source.
- **macOS path resolution** — tests comparing `pane_current_path` against `/tmp/...` allow both the literal and the `pwd -P` resolved form (`/private/tmp/...`) per §11.4.81 (XNU's `/tmp` symlink behaviour).
- **41_docs_user_guides_render.sh** added to P8 deliverables to satisfy `TMUX-CH-28` Challenge.
- New file: `scripts/tmx-state-bin` — compiled binary, committed alongside source. Operators can rebuild via `cd scripts/tmx-state && go build -o ../tmx-state-bin .` (also wired into `scripts/setup.sh`).